Lab 2 report

实验目的与内容

1. 实验目的：通过 Verilog 硬件描述语言实现一个可用的单周期 CPU ，本次实验将使用 Verilog 实现 CPU 的各个功能部件。
2. 实验内容：设计寄存器堆、ALU 及译码器数码管模块并且以此搭建一个简单的计算器。

逻辑设计

1. 各模块框图&数据通路：无
2. 状态机：无
3. 代码解释：

任务 1：寄存器堆设计（3 分）

请参考实验教程，设计**符合要求**的 32 位寄存器堆，并进行仿真。你需要在 Vivado 中创建一个项目，添加下面的仿真文件。

寄存器堆的仿真文件

在检查时，请向助教展示你的仿真结果，并介绍代码的设计思路。

代码:

```
module REG_FILE (
    input        [ 0 : 0]    clk,

    input        [ 4 : 0]    rf_ra0,
    input        [ 4 : 0]    rf_ra1,
    input        [ 4 : 0]    rf_wa,
    input        [ 0 : 0]    rf_we,
    input        [31 : 0]    rf_wd,

    output  reg  [31 : 0]    rf_rd0,
    output  reg  [31 : 0]    rf_rd1
```

```
);

    reg [31 : 0] reg_file [0 : 31];

    // 用于初始化寄存器
    integer i;
    initial begin
        for (i = 0; i < 32; i = i + 1)
            reg_file[i] = 0;
    end

    always @(posedge clk) begin
        if(rf_wa != 0) begin
            if(rf_we == 1)
                reg_file[rf_wa] <= rf_wd;
        end
    end

    always @(*) begin
        rf_rd0 = reg_file[rf_ra0];
        rf_rd1 = reg_file[rf_ra1];
    end
endmodule
```

本题中要求实现一个 32 位的寄存器，其中 wa、we、wd 分别为写地址、写使能和写数值端口，ra 和 rd 为读地址和读数值。首先初始化寄存器内容全部为 0，而后我们实现其读写功能，always 语句中的 if 判断条件

保证了 0 号寄存器永远为 0，同时对时钟上升沿到来时的使能信号进行了判断，实现写入功能。读功能即为简单的赋值即可。

任务 2：ALU 设计（3 分）

请参考实验教程，设计符合要求的 32 位 ALU，并进行仿真。你需要自行编写合适的仿真文件，验证每一种运算模式下 ALU 计算的正确性。一个可供参考的仿真框架如下：

ALU_tb

```
module ALU_tb();
    //...
    initial begin
        src0=32'hffff; src1=32'hffff; sel=5'H0;
        repeat(32) begin
            sel = sel + 1;
            #20;
        end
    end
    //...
endmodule
```

在检查时，请向助教展示你的仿真结果，并介绍代码的设计思路。

代码：module ALU (

```
    input        [31 : 0]        alu_src0,
    input        [31 : 0]        alu_src1,
    input        [ 4 : 0]        alu_op,

    output   reg     [31 : 0]        alu_res
);

`define ADD         5'B00000
`define SUB         5'B00010
`define SLT         5'B00100
`define SLTU        5'B00101
`define AND         5'B01001
`define OR          5'B01010
`define XOR         5'B01011
```

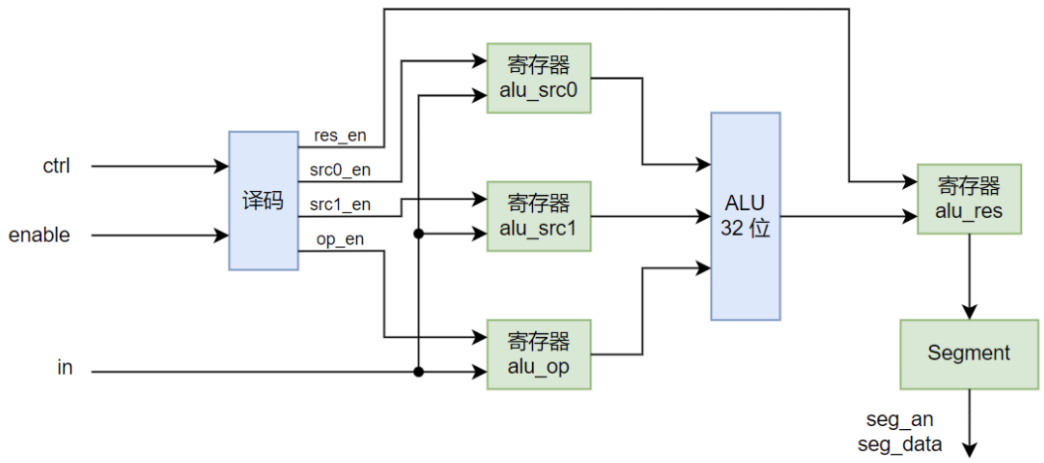
```
`define SLL                 5'B01110
`define SRL                 5'B01111
`define SRA                 5'B10000
`define SRC0                5'B10001
`define SRC1                5'B10010

    always @(*) begin
        case(alu_op)
            `ADD  :
                alu_res = alu_src0 + alu_src1;
            `SUB  :
                alu_res = alu_src0 - alu_src1;
            `SLT  :
                alu_res = ($signed(alu_src0) < $signed(alu_src1)) ? 32'd1 : 32'd0;
            `SLTU :
                alu_res = (alu_src0 < alu_src1) ? 32'd1 : 32'd0;
            `AND  :
                alu_res = alu_src0 & alu_src1;
            `OR   :
                alu_res = alu_src0 | alu_src1;
            `XOR  :
                alu_res = alu_src0 ^ alu_src1;
            `SLL  :
                alu_res = alu_src0 << alu_src1[4 : 0];
            `SRL  :
```

```
            alu_res = alu_src0 >> alu_src1[4 : 0];
        `SRA   :
            alu_res = $signed(alu_src0) >>> alu_src1[4 : 0];
        `SRC0 :
            alu_res = alu_src0;
        `SRC1 :
            alu_res = alu_src1;
        default :
            alu_res = 32'H0;
    endcase
  end
endmodule
```

按照题目所给框架，我们采用`define 语句定义各种操作，此方法可以提高代码的阅读性并且减少因读 01 串带来的失误。通过 case 语句进行判断并执行相应功能，在 default 中将 alu 输出结果归零，大部分操作的实现都采用了行为级描述。

任务 3：在线计算器（3 分）

请使用任务 2 中的 ALU，在 FPGAOL 上搭建一个简单的计算器。



本题的数据通路

代码：

```
module TOP (
    input       [ 0 : 0]    clk,
    input       [ 0 : 0]    rst,

    input       [ 0 : 0]    enable,
    input       [ 4 : 0]    in,
    input       [ 1 : 0]    ctrl,

    output      [ 3 : 0]    seg_data,
    output      [ 2 : 0]    seg_an
);

    reg     [31 : 0]    alu_src0;
    reg     [31 : 0]    alu_src1;
```

```
    reg       [ 4 : 0]          alu_op;
    reg       [31 : 0]          temp;
    wire      [31 : 0]          res;

ALU alu (
    .alu_src0(alu_src0),
    .alu_src1(alu_src1),
    .alu_op(alu_op),

    .alu_res(res)
);

Segment segment (
    .clk(clk),
    .rst(rst),
    .output_data(temp),
    .seg_data(seg_data),
    .seg_an(seg_an)
);

always @(*) begin
        if(enable) begin
            case(ctrl)
                2'B00:
                    alu_op = in;
```

```
                2'B01:
                    alu_src0[31:0] = {{27{in[4]}}, in[4:0]};
                2'B10:
                    alu_src1[31:0] = {{27{in[4]}}, in[4:0]};
                2'B11:
                    temp = res;
            endcase
        end
        if(rst) begin
            temp = 0;
        end
    end

endmodule
```

根据题给数据通路及相关要求，对于例化的 ALU 模块，我们接入的输入应该进行符号扩展，输出则无需改动；对于 ctrl 信号，当其有效时进入 case 语句判断 enable 信号的情况，对于前三种输入情况，我们对应将 in 的值赋给相应变量即可，最后将输出情况导入题目给出的 Segment 模块中并且在上板时输出显示。

任务 4：初始化存储器（1 分）

请创建一个新的项目，例化合适大小的存储器 IP 核（分布式、块式均可），将上一次实验生成的指令段 COE 文件导入到 IP 核中，并向助教展示。

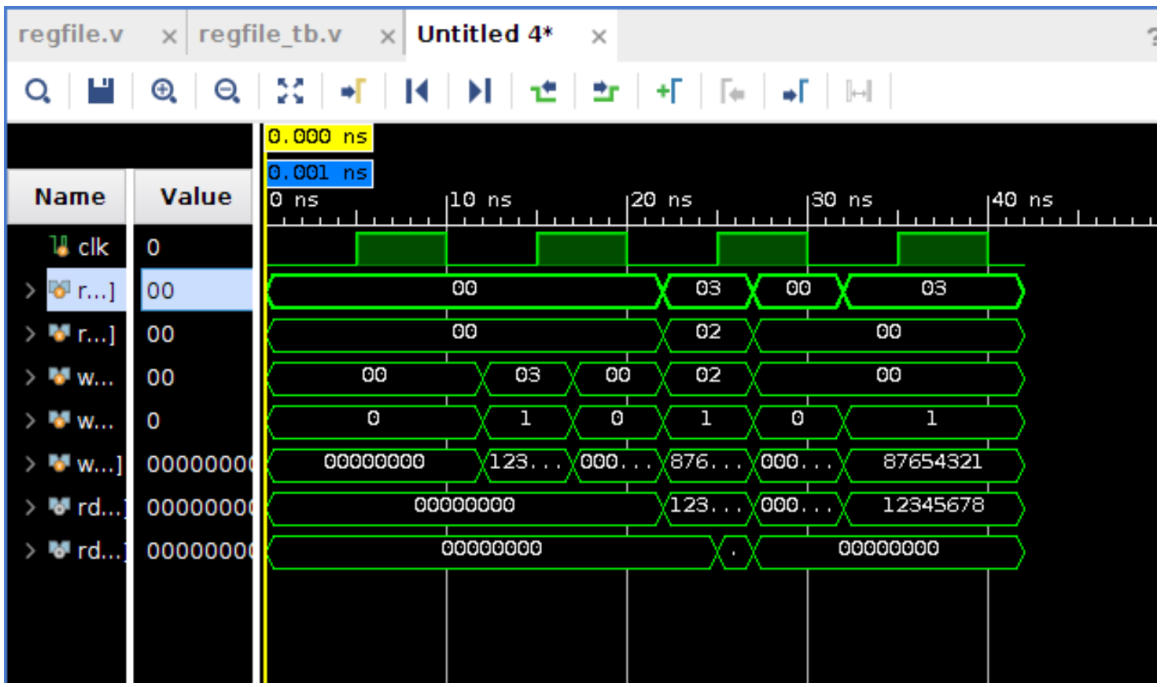
例化 Coe 文件情况如下：

```
module IP(
);

   dist_mem_gen_0 your_instance_name (
  .a(a),      // input wire [5 : 0] a
  .d(d),      // input wire [15 : 0] d
  .clk(clk),  // input wire clk
  .we(we),    // input wire we
  .spo(spo)  // output wire [15 : 0] spo
);
Endmodule
```

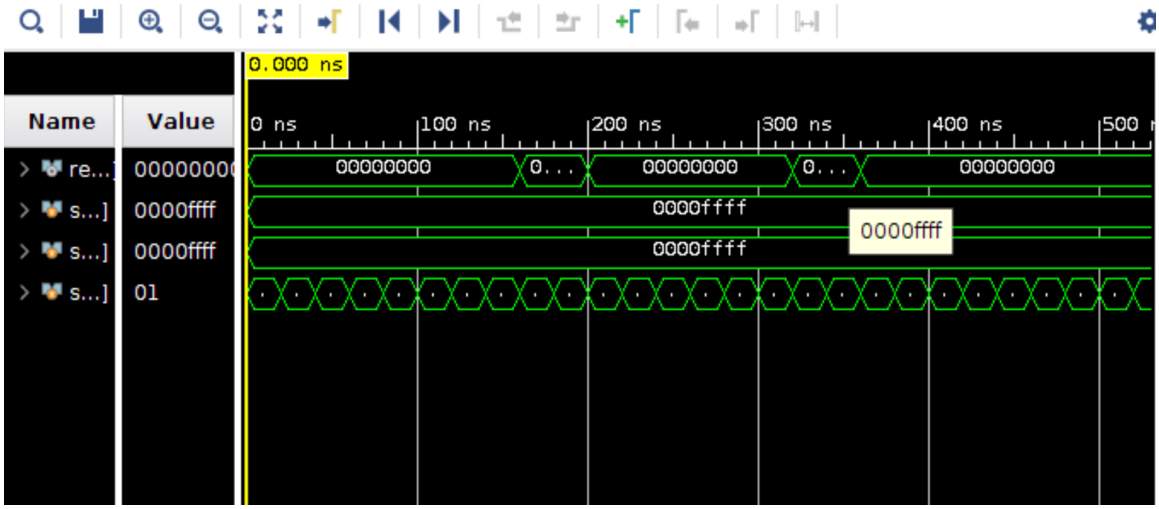
仿真结果与分析

1. 仿真结果：

   任务一仿真结果如下：



任务二实现的 ALU 模块由于直接在任务三的计算其中被例化，故其功能实现与任务三一并上板检查，此处仅为较简单的仿真波形。

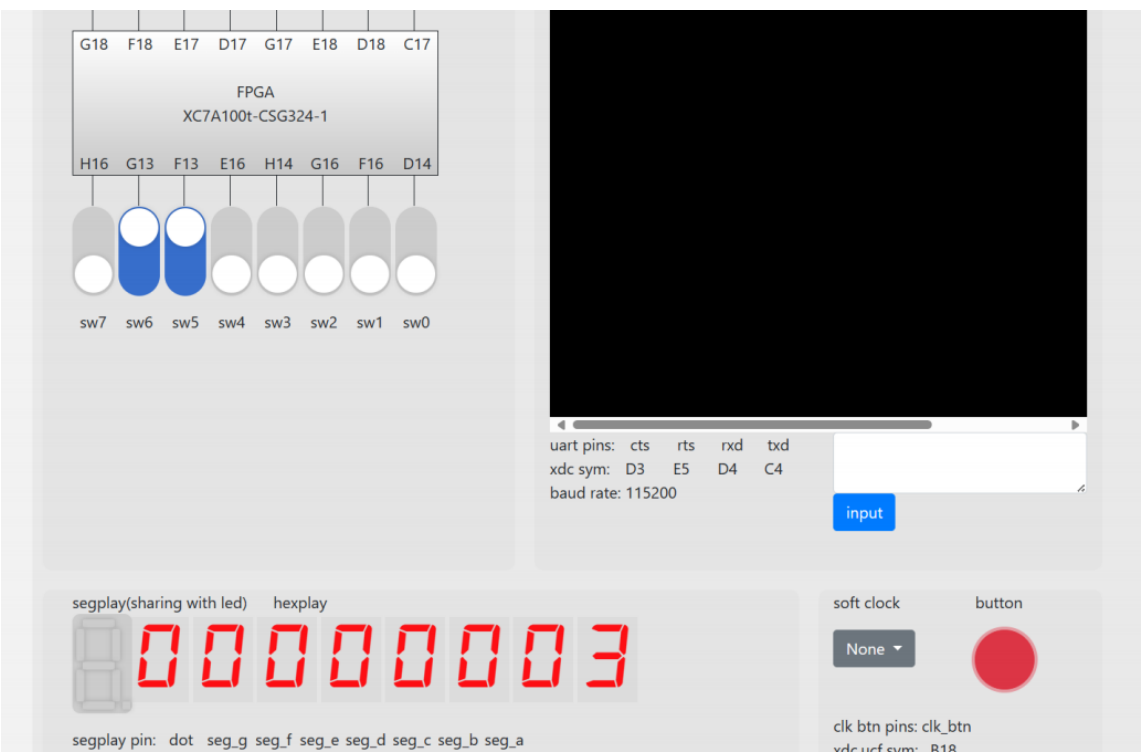


电路设计与分析

1. RTL 电路描述：略

测试结果与分析

1. 实验结果：
2. 任务一二皆可通过仿真波形检查功能，且任务三例化了任务二中的 ALU 模块，故此处仅展示任务三的结果
3. 任务三：



总结

1. 总结：本次实验帮助我复习了 verilog 的相关语法知识，具体为寄存器及 ALU 的功能实现，同时完成了 CPU 的部分组件，熟练阅读数据通路的能力。
2. 对老师和助教的建议：无。